IMPERIAL

Imperial College London
Department of Mathematics

Adaptive quantile boosted classification for imbalanced data

LAURYNE MOYSE

CID: 06009256

Supervised by ZAK VARTY

29 August 2025

Submitted in partial fulfilment of the requirements for the MSc in Statistics at Imperial College London

The work contained in this thesis is my own work unless otherwise stated.

Signed: LAURYNE MOYSE Date: 29 August 2025

Acknowledgements

First, I would like to thank my supervisor, Zak Varty, for his continual guidance and constructive feedback throughout this project. I am also grateful to the members of my research group for their enthusiastic support, which made this experience both productive and enjoyable.

Abstract

Imbalanced binary classification represents an important challenge in machine learning. Conventional models can fail to identify important events due to their rarity. We explore the use of the Smooth Quantile Boost Classification (SQBC) algorithm in this context. We adapt SQBC to imbalance by varying the quantile level parameter τ using hyperparameter tuning or gradient descent, and by introducing the smoothed asymmetric quantile loss used in SQBC as a custom objective function in XGBoost.

We evaluate these variants against state-of-the-art gradient boosting algorithms on various benchmark datasets, comparing the area under precision-recall curve (PR-AUC). Results show that tuning τ systematically improves PR AUC, particularly when the minority class represents less than 20% of the data. While standalone SQBC suffers from slower training and limited competitiveness, its integration into the XGBoost framework (XGB-SQBC) proves to be both fast and effective, frequently outperforming baseline models, suggesting that XGB-SQBC might be a competitive option for imbalanced classification.

Contents

1 Introduction . . . 3
2 Background . . . 5
2.1 Binary Imbalanced Classification . . . 5
2.2 Gradient Boosting . . . 8
2.3 Quantile Regression . . . 9
2.4 Boosted Quantile Regression . . . 10
3 Methods . . . 14
3.1 General . . . 14
3.2 Optimizing τ . . . 16
3.3 Smooth Quantile Boost Classification (SQBC) . . . 18
3.4 Customized XGBoost . . . 19
3.5 Adaptive SQBC . . . 20
4 Results . . . 23
4.1 Grid Search . . . 24
4.2 Adaptive SQBC . . . 25
4.3 Models Performance . . . 25
4.4 Models Efficiency . . . 27
5 Discussion . . . 28
6 Endmatter . . . 30
7 Supplementary material . . . 33

1 Introduction

Binary classification is a subject in a wide variety of domains such as fraud detection, churn prediction, or medical diagnosis (West and Bhattacharya (2016), Burez and Van den Poel (2009) and Murtaza et al. (2020)). In these fields, the goal is to detect or predict an event of interest. Most of the time, this event occurs rarely (less than 10% of the time) and therefore represents the minority class. These problems often involve asymmetric classification costs, where missing a positive example is costly compared to missing a negative example. For example, in the medical field, a false negative implies failing to detect a disease and depriving a patient of receiving appropriate treatment, which can have serious consequences, while a false positive rather leads the patient to perform unnecessary additional tests. The development of algorithms able to learn a good representation of the data and to accurately detect rare events, even when trained with a few portion of examples, is therefore crucial in the machine learning field (Weiss (2004)).

In the case of extreme imbalance, resampling methods present a good alternative, especially SMOTE proposed by Chawla et al. (2002). To help alleviate sample scarcity, SMOTE generates new instances of synthetic minority classes rather than simply duplicating existing ones, unlike other sampling methods. Although effective, these techniques have their limits, highlighted by Ranjan (2025): they can alter the original data distribution, they present a risk of overfitting if the minority class is really scarce, and they are limited in high dimension. In this paper, we propose to focus on machine learning methods which don't require to modify the original samples, and more specifically, on gradient boosting algorithms.

Boosting methods are part of ensemble learning, a family of techniques which combine multiple models, called "base learners", into a final predictor. The goal is to obtain, via aggregation, a more robust model with reduced variance and bias.

Among the best-known ensemble methods is bagging (bootstrap aggregating), of which Random Forest is a popular implementation, and which trains learners in parallel on bootstrapped subsets of the data, before aggregating them. Another example is stacking, which trains a meta-model on the predictions of several base learners. Finally, boosting methods consist in sequentially combining weak learners (predictors with accuracy barely above chance), typically small decision trees called "stumps". These learners are trained to successively correct the errors of the previous ones, progressively focusing on the most difficult cases.

Gradient boosting, originally developed by Friedman (2001), extends this idea by fitting new learners to the negative gradient of a loss function, performing gradient descent in function space. These methods have gained popularity over the past few years due to their high predictive accuracy, computational efficiency, scalability, and flexibility across diverse tasks, even with few hyperparameter tuning (Bentéjac et al. (2021)). Researchers have developed several popular frameworks of gradient boosting, such as CatBoost by Prokhorenkova et al. (2018), LightGBM by Ke et al. (2017), and XGBoost by Chen and Guestrin (2016) which are widely used in practice.

Most originally, Zheng (2010) introduced the quantile boost (QBoost) algorithm, that combines quantile regression, presented by Koenker and Hallock (2001), and boosting methods. In a follow-up, Zheng (2011) proposed a smooth approximation of the quantile loss, addressing the non-differentiability of the function, therefore enabling gradient-based optimization, but restricted to regression tasks. Building on this work, Wang and Ye (2016) developed the Smooth Quantile Boost Classification (SQBC) algorithm, a gradient boosting method for classification incorporating this smooth quantile loss. They demonstrated that SQBC outperformed classical boosting algorithms of the time in several classification settings. SQBC is particularly interesting because quantile regression naturally introduces asymmetry into

the prediction process and can allow the model to focus on the minority class and to better identify rare cases. In Wang and Ye (2016), the SQBC algorithm is used in a balanced setting, with a quantile level parameter τ fixed to the median value 0.5.

This work explores the use of boosted quantile regression for imbalanced binary classification. We focus on the role of the quantile level τ and adopt various approaches, such as tuning and learning, to adapt it to data asymmetry and see its impact. Building upon the SQBC framework, we develop and implement several methods, and evaluate their performance against state-of-the art boosting algorithms on multiple benchmark datasets. The report is structured as follows. Section 2 provides the necessary background to understand and describe the SQBC framework, highlighting our contributions and objectives. Section 3 details the implementation process, including the algorithms and datasets used, providing explanations and justifications of the choices made. Section 4 presents and analyzes the results. Finally, Section 5 discusses the findings, summarizes the work, and outlines limitations and potential directions for future research.

2 Background

2.1 Binary Imbalanced Classification

Classification consists in learning a function $f : \mathbb{R}^d \to \mathcal{Y}$ from a design matrix $D \in \mathbb{R}^{n \times d}$, composed of samples $x_i \in \mathbb{R}^d$, to a discrete target variable Y. In the context of binary classification, $\mathcal{Y} = \{0, 1\}$, typically representing application-specific labels such as “fraud” vs “not fraud”.

Given a classification outcome, we divide the predictions into four categories: false positives (FP), false negatives (FN), true positives (TP), and true negatives (TN).

Various metrics are used to evaluate the performance of a classification model. The most common is accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

which measures the proportion of correctly predicted samples. However, in imbalanced settings, accuracy can be misleading as a model can obtain a high accuracy by predicting only the majority class and failing to identify the minority class (Weiss (2004)).

Therefore, class-specific metrics like recall:

$$Recall = \frac{TP}{TP + FN}$$

and precision:

$$Precision = \frac{TP}{TP + FP}$$

or composite metrics like F_β-score:

$$\frac{(1 + \beta^2) \cdot Precision \cdot Recall}{\beta^2 \cdot Precision \cdot Recall}$$

are often preferred, as they emphasize the ability to identify the minority class.

Classification models usually output probabilities, which are converted to binary predictions via a decision threshold. This threshold is often fixed to a standard value (0.5) but it is possible to adapt it in order to optimize a specific metric (e.g., f1-score, recall).

Threshold-independent metrics, based on areas under curves, are also used. The ROC AUC (Area Under the Receiver Operating Characteristic Curve) measures the area under the curve of the True Positive Rate TPR (recall) against the False Positive Rate FPR ($\frac{FP}{FP+TN}$). The PR AUC (Area Under the Precision–recall Curve) measures the area under the curve of recall against precision. We seek to obtain the largest possible areas.

Most of the time, as recall increases (the model identifies more true positives), precision typically decreases (more false positives are introduced). The PR curve directly shows the trade-off between detecting more positives while avoiding too many false alarms, not considering the true negative quarter. It is therefore particularly relevant in the context of imbalance classification (Sofaer et al. (2019)). This is not the case for the ROC curve, as it is possible to achieve a low FPR despite the imprecision of the prediction if the negative examples are consequently more numerous than the positive examples.

In practice, the choice of evaluation metrics to optimize depend on the specific context and objectives. For example, in identity verification or fraud detection, maintaining a low FPR is crucial, even at the expense of a lower TPR, since false alarms are costly. In contrast, in medical diagnosis, prioritizing a high TPR is often more important, even if it implies a higher FPR.

Furthermore, no single metric captures all aspects of model's performance and it is better to consider common metrics together in order to gain a comprehensive understanding.

All of these metrics are typically computed for model evaluation after the training. However, using tailored loss functions or adjusting decision thresholds can be used to align model predictions with specific objectives (Hernández-Orallo et al. (2012)).

2.2 Gradient Boosting

Gradient boosting learns a classification or regression model through iterative gradient descent in function space. In our case, let (X, Y) be a generic pair of random variables, with $X \in \mathbb{R}^d$ and $Y \in \{0, 1\}$, and the training data $\{(x_i, y_i)\}_{i=1}^n$. The goal is to find a predictor f that minimizes the expected value of a differentiable loss function $\mathcal{L}$:

$$f^* = \arg\min_f \mathbb{E}_{X,Y}[\mathcal{L}(Y, f(X))].$$

In a regression context, losses such as the mean squared error are convex with respect to the model output $f(x)$, ensuring convergence of the descent to the global minimum. In classification, the final decision function $\mathbf{1}(f(x) > \text{threshold})$ is non-differentiable and non-convex. However, models can be trained on convex losses such as cross-entropy (Wang et al. (2022)), producing a continuous output. The threshold is then applied post-training to convert predicted scores into class labels.

At each iteration m, the gradient of the loss function with respect to the current prediction is computed:

$$U_i^{(m)} = \frac{\partial}{\partial f^{(m-1)}(x_i)} \mathcal{L}(y_i, f^{(m-1)}(x_i)).$$

A weak learner $g^{(m)}(x)$ is then fitted to the negative gradient, called the "pseudo-residuals", to approximate $x_i \mapsto U_i^{(m)}$.

The model, initialized at the beginning of the descent by $f^{(0)}(\cdot)$, is updated as:

$$f^{(m)}(x) = f^{(m-1)}(x) + \eta_m * g^{(m)}(x)$$

where η_m is a hyperparameter known as the "learning rate", defining the contribution of each learner.

The final model is obtained when a previously defined number M of iterations is reached.

2.3 Quantile Regression

In classic linear regression, given a continuous target variable Z and covariates X, the goal is to estimate the conditional mean $\mathbb{E}(Z|X)$. In contrast, quantile regression estimates the τ-th conditional quantile:

$$Q_\tau(Z|X = x) = \inf\{z \in \mathbb{R} : \Pr(Z \leq z \mid X = x) \geq \tau\} \tag{1}$$

with $0 \leq \tau \leq 1$, the "quantile level".

We model the conditional quantile as a function of the covariates: $Q_\tau(Z|X = x) = f_\tau(x)$ and try to solve:

$$f^* = \arg\min_f \sum_{i=1}^{n} \rho_\tau(z_i - f_\tau(x_i))$$

where:

$$\rho(u) = u[\tau - \mathbf{1}(u < 0)] \tag{2}$$

is the quantile or pinball loss, also called the "check function".

Quantile regression therefore introduces an asymmetric loss function, which penalizes positive and negative residuals of same modulus differently, associating different costs depending on the sign, as illustrated in Figure 1.

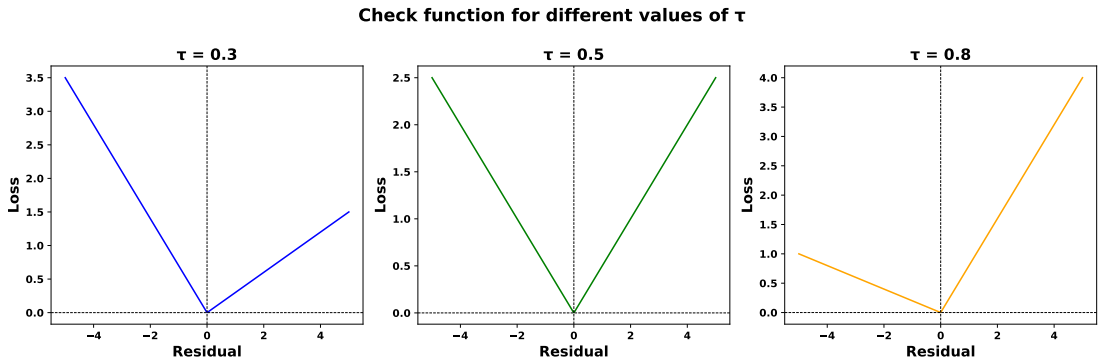

Figure 1: Representation of check function for various quantile levels $\tau \in \{0.3, 0.5, 0.8\}$ as a function of residual (u=z-f(x)).

2.4 Boosted Quantile Regression

2.4.1 Boosted Quantile framework

In Zheng (2010) quantile-boosted framework, a key assumption is the existence of a latent continuous variable Y^* modeling a richer underlying process:

$$Y^* = h(x) + \epsilon, \;\; Y = \mathbf{1}(Y^* \geq 0)$$

where $h(\cdot)$ is the true underlying signal and ϵ is a noise term with zero mean.

Since we don't have access to Y^*, the idea is to estimate a conditional quantile $Q_\tau(Y^*|X)$. We assume that this quantile can be written as a function $f(x, \beta)$, where β is the parameter vector.

By the definition of a quantile given in Equation (1):

$$P(Y^* \geq f(x, \beta)|x) = 1 - \tau. \tag{3}$$

Additionally, since $Y = \mathbf{1}(Y^* \geq 0)$, we have:

$$P(Y = 1|x) = P(Y^* \geq 0|x). \tag{4}$$

Combining (3) and (4), we obtain:

$$P(Y = 1|x) \begin{cases} > 1 - \tau & \text{if } f(x, \beta) > 0 \\ = 1 - \tau & \text{if } f(x, \beta) = 0 \\ < 1 - \tau & \text{if } f(x, \beta) < 0 \end{cases}$$

Hence, deciding to assign $Y = 1$ whenever the predicted probability exceeds the limit $1 - \tau$, called the "decision threshold", we derive the following rule:

$$\hat{Y} = \mathbf{1}(P(Y = 1|x) \geq 1 - \tau) = \mathbf{1}(f(x, \hat{\beta}) \geq 0) \tag{5}$$

where $\hat{\beta}$ is the estimation the vector parameter β .

Due to the presence of an indicator function, the quantile loss given in Equation (2) is not differentiable on all its definition set, making gradient descent described in Section 2.2 impossible. Zheng (2011) therefore introduced a smooth version of the check function:

$$S_{\tau,\alpha}(x) = \tau x + \alpha \log(1 + \exp(-x/\alpha)) \tag{6}$$

where $\alpha > 0$ is called the smooth parameter. The smaller the value of α, the closer the smooth approximation is to the original check function.

2.4.2 SQBC algorithm

Assume $\mathcal{G}$ is the collection of all weak learners, i.e., $\mathcal{G} = \{g_1(x), g_2(x), \ldots, g_d(x)\}$, where d is the total number of weak learners, possibly $d = \infty$. Denote the space spanned by these weak learners as

$$\mathcal{L}(\mathcal{G}) = \left\{ \sum_{i=1}^{d} c_i g_i(x) \,\middle|\, c_i \in \mathbb{R},\, i = 1, \ldots, d \right\}.$$

We assume $f(x, \beta)$ lies in the space spanned by the functions in $\mathcal{G}$, i.e., $f(x, \beta) \in \mathcal{L}(\mathcal{G})$.

Within the framework describe in Section 2.4.1, the SQBC algorithm proposed in Wang and Ye (2016) aims to estimate the class label $Y = \mathbf{1}(f(x, \beta) \geq 0)$ by minimizing the following loss function:

$$l(Y, f) = S_{\tau,\alpha} \left[Y - \mathbf{1}(f(x, \beta) \geq 0) \right].$$

Once again, to keep the loss function differentiable, Zheng (2011) and Wang and Ye (2016) use a smooth version of the indicator function $\mathbf{1}(t \geq 0)$: $K\left(\frac{f(x,\beta)}{h}\right)$.

The smooth function $K(\cdot)$ has to have the following properties (Kordas (2006)):

$$K(t) > 0,\ \forall t \in \mathbb{R}, \lim_{t \to \infty} K(t) = 1, \lim_{t \to -\infty} K(t) = 0,$$

and h is a small positive value.

In this paper, following Wang and Ye (2016), we take $K(\cdot)$ as the standard normal

cumulative distribution function:

$$\Phi(z) = \int_{-\infty}^{z} \phi(t)dt \text{ with } \phi(z) = \frac{1}{\sqrt{2\pi}} \exp(-z^2/2).$$

As a result, the loss to minimize becomes:

$$l(Y, f) = S_{\tau,\alpha}\left[Y - K\left(\frac{f(x,\beta)}{h}\right)\right].$$

Substituting (6), it yields:

$$l(Y, f) = \tau * \left[Y - K\left(\frac{f(x,\beta)}{h}\right)\right] + \alpha * \log\left[1 + \exp\left(\frac{-(Y - K(\frac{f(x,\beta)}{h})}{\alpha}\right)\right] \quad (7)$$

which is differentiable with respecto to f.

Applying the generic functional gradient descent algorithm to the loss given in (7) yields the SQBC method described in Algorithm 1.

Wang and Ye (2016) showed that this method obtained better accuracy than standard boosting algorithms AdaBoost (Freund and Schapire (1997)), Logit-Boost (Friedman et al. (2000)) and L2Boost (Bühlmann and Yu (2003)), in high-dimensional settings and exhibited increased robustness to outliers. But no focus was made on class imbalance and τ was fixed to 0.5. However, as illustrated in Figure 1, changing τ leads to an asymmetric loss function which can potentially adapt to different degree of class imbalance.

Algorithm 1 Smooth Quantile Boost Classification (SQBC)

Require: Training data $\{(x_i, Y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^p$, $Y_i \in \{0, 1\}$, smooth parameter α, h, quantile level τ, number of iterations M, step size η

Initialize $\hat{f}^{(0)}(x) \leftarrow 0$

for $m = 1$ **to** M **do**

Compute negative gradient $-\frac{\partial l(Y,f)}{\partial f}$ and evaluate at $\hat{f}^{(m-1)}(x)$:

$$U_i = -K'\left(\frac{\hat{f}^{(m-1)}(x_i)}{h}\right) \cdot \left[\left(1 + \exp\left(\frac{Y_i - K(\hat{f}^{(m-1)}(x_i)/h)}{\alpha}\right)\right)^{-1} - \tau\right]$$

with $i = 1, \ldots, n$

Fit base learner $\hat{g}^{(m)}(x)$ to data $\{(x_i, U_i)\}_{i=1}^n$

Update estimate:

$$\hat{f}^{(m)}(x) \leftarrow \hat{f}^{(m-1)}(x) + \eta \hat{g}^{(m)}(x)$$

end for

Output: classifier $\mathbf{1}(\hat{f}^{(M)}(x) \geq 0)$

3 Methods

3.1 General

3.1.1 Models and Parameters

We use multiple state-of-the-art (SOTA) models, namely three gradient boosting algorithms: XGBoost, CatBoost, and LightGBM. To ensure a fair comparison and assess their discriminative power, these models, along with the customized ones described in further sections, are all implemented with their default parameter settings. The only exception is the quantile level τ, which is central to our research.

For SQBC, the parameters are set as follows:

- $\eta = 0.1$, chosen to be sufficiently small to ensure stable convergence
- $h = 0.1$ (Wang and Ye (2016))
- $\alpha = 0.5$ (Zheng (2011)), providing a balance between a close approximation to the check function and improved smoothness
- number of estimators: $M = 100$, chosen both for computational constraints and to match the SOTA baseline parameters
- the base learners are set to decision trees for all models, a choice widely adopted in SOTA boosting frameworks as trees are able to capture complex relationship in the data.

3.1.2 Data

We work on both real and generated data. The goal in the synthetic datasets is to control the proportion p of positive examples. The generation process is given by:

$$\begin{cases} X_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \mathbb{R}^d), W \sim \mathcal{N}(0, \mathbb{R}^d), \\ z_i = X_i^T W + \varepsilon_i, \varepsilon_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1), \\ \text{threshold} = \text{quantile}_{1-p}(\{z_i\}_{i=1}^n), \\ Y_i = \mathbf{1}(z_i \geq \text{threshold}) \end{cases}$$

In summary, we generate from a Gaussian model with noise. We impose an empirical threshold to control the portion of positive examples. It allows us to vary p and systematically compare models performance across different imbalance levels. We generate synthetic datasets for $p \in [0.001, 0.01, 0.05, 0.1, 0.5]$.

The real datasets are the following (p is the proportion of positive examples, n the number of samples and d the number of covariates):

- "sheart": Heart Disease Dataset (Kaggle): $p = 0.55$, $n = 303$, $d = 13$, indicating the presence of heart disease in a patient
- "churn": Employee Churn Data (Kaggle): $p = 0.20$, $n = 9799$, $d = 20$, organizational data of employee information
- "heart": Heart Disease Health Indicators Dataset (Kaggle): $p = 0.05$, $n = 10000$, $d = 21$, survey responses from 2015
- "fraud": Fraud Detection Dataset (Data Science Challenge): $p = 0.001$, $n = 80000$, $d = 27$, credit card frauds from transactions of a European bank in 2023 between 17/09 and 17/10.

These datasets were selected for their realistic use cases, variety in imbalance levels, feature types (categorical and continuous), and noise levels, which allow models to be evaluated and compared in various scenarios.

3.2 Optimizing τ

Standard boosting algorithms do not directly produce a binary outcome, but a probability for each sample to belong to a given class. The final classification is then obtained by thresholding this probability. By default, the decision threshold is set to 0.5, but it is possible to carry out threshold optimization after training in order to align the predictions with a specific metric.

In the SQBC framework, we introduced the decision rule: $\mathbf{1}(f(x, \beta) \geq 0)$. In practice, 0 could be replaced by any real value t, translating the decision boundary in a way that adapts to data characteristics, similarly to adjusting a probability threshold in a standard gradient boosting classifier.

As seen in Equation (5), t and τ can play similar roles in the sense that both modify the decision boundary. Nevertheless, τ acts in the latent space during training and intervenes in the pseudo-residuals base learners are trained on. In contrast, t is a post-training decision threshold applied on fixed output predictions, without affecting the learned parameters.

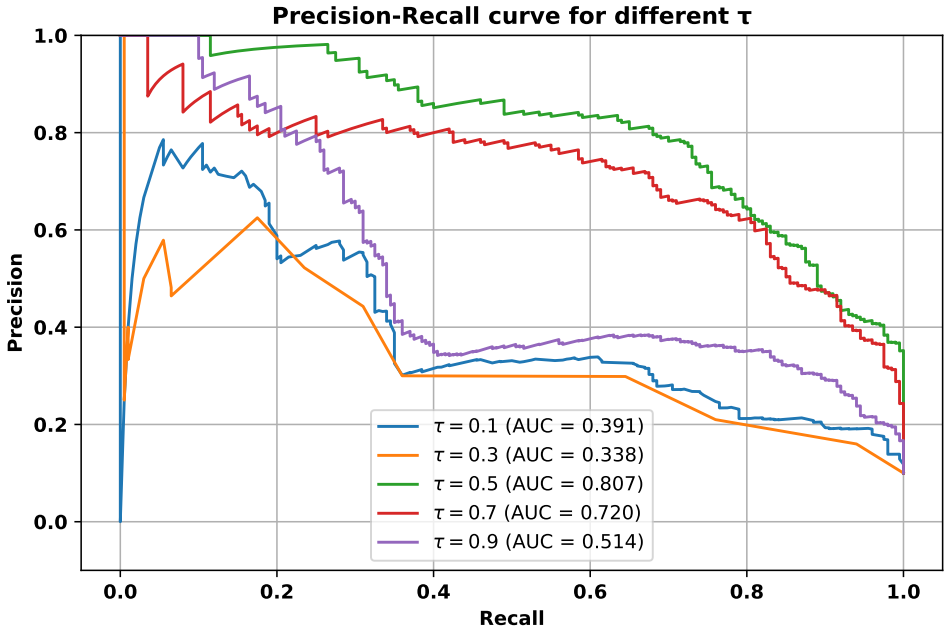

Figure 2: Representation of Precision-Recall curve for various quantile levels $\tau \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$.

The fact that τ appears in the loss opens the possibility, at least in principle, of learning this parameter jointly with f during training. Modifying τ during training can cause the model to learn how to better separate the classes according to the specified asymmetry.

On Figure 2, we notice that the PR curves are visibly distinct for different values of the quantile level τ. Since, the PR curve is threshold-agnostic, this result shows

that varying τ in SQBC actually affects the learning beyond simply shifting the decision threshold.

In the following sections, we describe the different model variants we implemented to control the value of τ.

3.3 Smooth Quantile Boost Classification (SQBC)

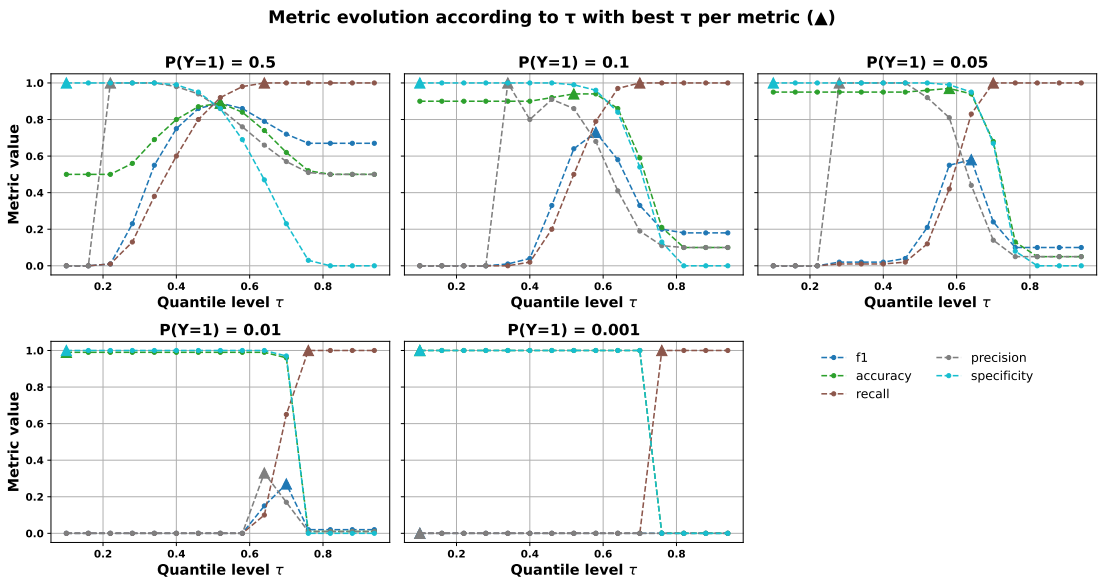

Figure 3: Evolution of various classification metrics according to τ at different imbalance degrees $p \in \{0.5, 0.1, 0.05, 0.01, 0.001\}$.

We implement the SQBC algorithm with three different strategies:

- fixing all parameters before training, with τ kept constant at 0.5
- optimizing τ via grid search with cross validation and interpolation. The smooth and continuous variation of performance metrics with τ observed in Figure 3 makes interpolation meaningful

- learning τ during training with the gradient descent described further in Section 3.5.

3.4 Customized XGBoost

The XGBoost framework presents several useful optimizations, such as regularization methods, that make it an efficient implementation of gradient boosting (Chen and Guestrin (2016)). To take advantage of these features, we implement the SQBC loss as a customized objective function in XGBoost, leading to the XGB-SQBC algorithm. For this, it is necessary to specify the gradient and Hessian of the loss function with respect to the model f because XGBoost performs a second-order Taylor expansion to compute the local updates.

The gradient is given in Algorithm 1:

$$-U_i = -K'\left(\frac{\hat{f}^{(m-1)}}{h}\right)\left[\left(1+\exp\left(\frac{Y_i - K(\hat{f}^{(m-1)}/h)}{\alpha}\right)\right)^{-1} - \tau\right]. \tag{8}$$

And the Hessian writes as:

$$\frac{\partial^2}{\partial f^2} l(Y,f) = \frac{f}{h^3}\phi\left(\frac{f}{h}\right)\left[\tau - \frac{1}{1+L}\right] + \frac{1}{\alpha}\left(\frac{f}{h}\phi\left(\frac{f}{h}\right)\right)^2 \frac{L}{(1+L)^2}$$

with $L = \exp\left((Y - K(\frac{f}{h}))/\alpha\right)$.

In practice, we observed that the Hessian presents local instabilities and can become negative in some region, as the loss is not convex with respect to f. The gradient descent in XGBoost requires the Hessian to be positive. We therefore

replaced it with a positive floor:

$$\tilde{H}(f) = \max(H(f), \varepsilon)$$

where $\varepsilon > 0$ is a small constant. This approximation may impair the accuracy of the model but allows us to stabilize the optimization while preserving the gradient direction.

We implemented this customized XGBoost using the first two strategies described in the previous section: keeping $\tau = 0.5$ and fine-tuning τ via grid search.

3.5 Adaptive SQBC

We propose a new SQBC algorithm where τ is learned automatically during training via loss optimization. This approach allows to avoid the costly cross-validation process induced by grid search. The goal is to offer an adaptive method that does not require to manually pre-specify the value of τ.

In order to keep τ between 0 and 1, we parametrize it as:

$$\tau = \sigma(\theta) = \frac{1}{1 + \exp(-\theta)} \tag{9}$$

where $\theta \in \mathbb{R}$.

We seek to compute the gradient of the loss with respect to θ, which is given by:

$$\frac{\partial l}{\partial \theta} = \frac{1}{n} \left(\sum_{i=1}^{n} \frac{\partial l}{\partial f_i} \frac{\partial f_i}{\partial \tau} + \frac{\partial l}{\partial \tau} \right) \cdot \frac{d\tau}{d\theta}. \tag{10}$$

We already know from Equations (9) and (7):

$$\frac{d\tau}{d\theta} = \tau(1-\tau) \tag{11a}$$

$$\frac{\partial l}{\partial f_i} = -U_i \tag{11b}$$

$$\frac{\partial l}{\partial \tau} = Y - K\left(\frac{f}{h}\right) \tag{11c}$$

Now, $\frac{\partial f_i}{\partial \tau}$ depends on the class of f. At a given iteration m, a base learner $g^{(m)}$ is fitted on the residuals $U_i^{(m)}$. If we perform an OLS at each step, and $x \in \mathbb{R}^d$ is a line of the design matrix D, we have an explicit expression:

$$\hat{g}^{(m)}(x) = x\hat{\beta}_\tau^{(m)} \tag{12a}$$

$$\hat{\beta}_\tau^{(m)} = (D^T D)^{-1} D^T U_\tau^{(m)} \tag{12b}$$

And: $\hat{f}^{(m)}(x) = \hat{f}^{(m-1)}(x) + \eta\hat{g}^{(m)}(x)$, which means:

$$\frac{\partial}{\partial \tau}\hat{f}^{(m)}(x) = \frac{\partial}{\partial \tau}\hat{f}^{(m-1)}(x) + \eta\frac{\partial}{\partial \tau}\hat{g}^{(m)}(x) \tag{13}$$

We have:

$$\frac{\partial}{\partial \tau}\hat{g}^{(m)}(x) = x\frac{\partial}{\partial \tau}\hat{\beta}_\tau^{(m)} = H\frac{\partial}{\partial \tau}U_\tau^{(m)} \tag{14}$$

where $H = x(D^T D)^{-1} D^T$ (Equation (12b)).

$U^{(m)}$ depends on $\hat{f}^{(m-1)}$. If we ignore the backpropagation and only consider the effect of τ on f at step m (i.e., the value of τ does not affect the predictions constructed at previous iterations), we can assume:

$$\frac{\partial}{\partial \tau}\hat{f}^{(m-1)}(x) = 0 \tag{15}$$

We obtain from Equations (13) and (8):

$$\frac{\partial}{\partial \tau}\hat{f}^{(m)}(x) = \eta \frac{\partial}{\partial \tau}\hat{g}^{(m)}(x)$$
$$\frac{\partial}{\partial \tau}U_{\tau}^{(m)} = K'\left(\frac{\hat{f}^{(m-1)}(x)}{h}\right)$$

Combined with Equation (14), this yields:

$$\frac{\partial}{\partial \tau}\hat{f}^{(m)}(x) = \eta H K'\left(\frac{\hat{f}^{(m-1)}(x)}{h}\right) \tag{16}$$

Substituting Equations (11) and (16) in Equation (10), the final gradient is:

$$\frac{\partial}{\partial \theta} l(Y, \hat{f}^{(m)}(x)) =$$
$$\frac{1}{n}\left[\sum_i -U_{\tau}^{(m)} * \eta H K'\left(\frac{\hat{f}^{(m-1)}(x)}{h}\right) + Y - K\left(\frac{\hat{f}^{(m)}(x)}{h}\right)\right] \cdot \tau^{(m-1)}(1-\tau^{(m-1)}). \tag{17}$$

The approximation stated in (15) is a simplification and could be significantly inaccurate. But, modeling the full dependency via previous iterations of the loss on τ would be complex and computationally greedy.

Now, we consider several methods to approximate the derivative of the loss with respect to τ:

- Method 1 (Naive gradient): From Equation (7), simply write:

$$\frac{\partial l}{\partial \theta} = \frac{\partial l}{\partial \tau}\frac{d\tau}{d\theta} = (Y - K(f))(1-\tau)\tau.$$

by totally ignoring the dependence of f on τ. This is a rough approximation but it presents the practical advantage that any base learner can be used, since it does not require differentiability of f.

- Method 2 (Partial gradient): Write $\frac{\partial l}{\partial \theta}$ as derived in Equation (17), this method accounts for the dependence of $f^{(m)}$ on τ. Although still incomplete, this approach is more rigorous. However, it requires the base learner to be explicitly differentiable in order to compute $\frac{\partial f}{\partial \tau}$, and linear regressors in this specific case.

The second method is certainly more accurate, but as we can see in Equation (17), it requires a matrix calculation which involves the design matrix D. This makes it much slower than the first method, and in high dimension, this calculation can quickly become very expensive or even infeasible. In the results section, the performances of the two methods are compared, and only method 1 is applied to the fraud dataset which has 80,000 rows and could present scalability issues.

After preliminary experiments of the gradient descent SQBC algorithm, we initialize the parameter $\theta = 0$, which represents a neutral starting point at the median $\tau = 0.5$. The learning rate for gradient descent updates is set to $lr_\theta = 0.1$ and is chosen to balance convergence speed and stability.

4 Results

Throughout this study, and for the reasons outlined in Section 2.1, we focus on evaluating and ranking models based on the PR AUC score. This metric directly reflects the model's ability to accurately identify positive examples.

We only present the most relevant results in this section, but the full pipeline output is available in the supplementary material.

Model names are defined as follows: models labeled "gs" mean that a grid search was performed, "XGB-SQBC" refers to the XGBoost framework using the SQBC-customized loss, "Grad M1" and "Grad M2" refer to the SQBC algorithm with gradient descent used respectfully with naive gradient and partial gradient (method 1 and method 2 described in Section 3.5).

4.1 Grid Search

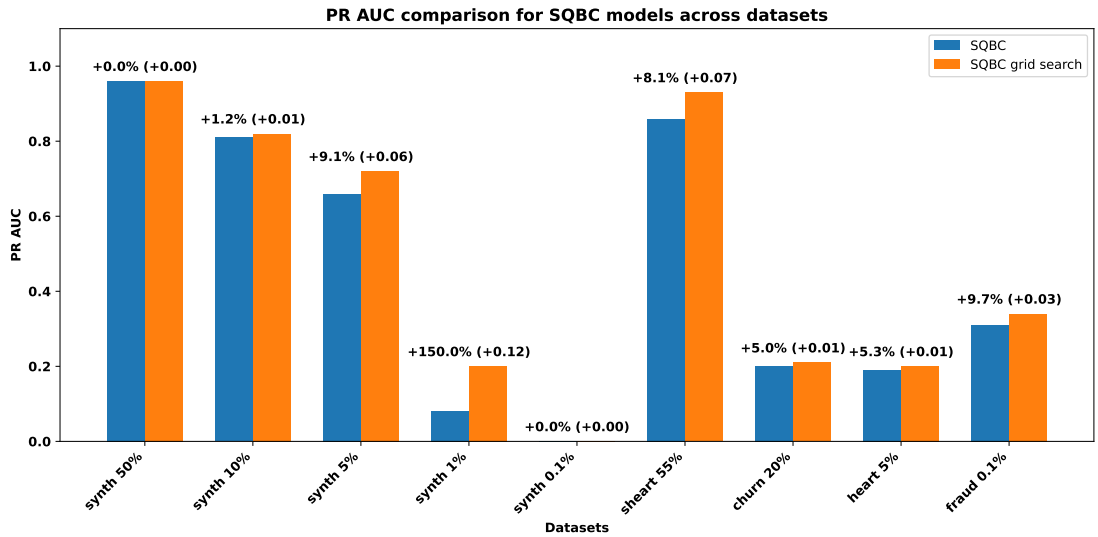

Figure 4: PR AUC values for SQBC model with and without grid search on τ across benchmark datasets, with percentage and points of PR AUC gained performing grid search.

The tuning of the hyperparameter τ with grid search always improve the PR AUC across datasets.

We can observe in Figure 4 that the difference is particularly noticeable on moderately imbalanced datasets, where the PR AUC improve substantially when τ gets closer to 1 due to grid search.

On more balanced datasets, or on datasets that are inherently very challenging, (e.g., due to a weak signal-to-noise ratio or extreme imbalance), the improvement becomes marginal or even negligible as all models perform really poorly.

4.2 Adaptive SQBC

We compared the naive (M1) and partial gradient (M2) methods using OLS as the base learner to avoid bias due to data structure, particularly on the synthetic datasets which presents a strong linear dependence. In this setting, "Grad M1" and "Grad M2" achieve identical PR AUC across all datasets. This suggests that the additional complexity of Method 2 does not translate into practical performance gains compared to Method 1. As discussed in Section 3.5, since Method 2 is computationally more demanding, we retain only Method 1 in the following experiments, this time using decision tree base learners. The equivalence observed between the two methods justifies the choice of M1 as an approximation also in the tree-based setting.

4.3 Models Performance

We present in Figure 5 the top three models for each dataset based on PR AUC.

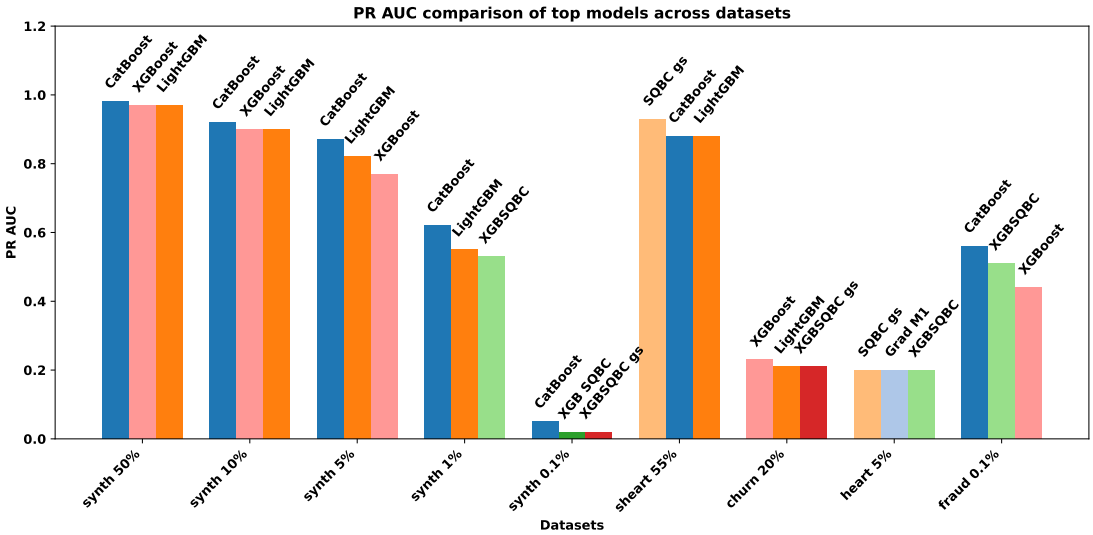

Figure 5: Top 3 PR AUC obtained by tested models on benchmark datasets.

Regarding synthetic datasets, standard gradient boosting models (CatBoost, XGBoost, LightGBM) systematically rank in the top 3 models at higher positive class proportions (from 50%, to 5%). When the imbalance increases, the advantage progressively shifts toward SQBC variants.

Regarding real data, for balanced dataset (p=55%, "sheart"), standard boosting methods perform well. But similarly to synthetic dataset, as the imbalance increases, SQBC variants can compete or even surpass SOTA gradient boosting methods, which demonstrates the potential benefit of incorporating τ into the loss to adjust for class imbalance.

We can notice that the best SQBC model in the top 3 is most often the XGB-SQBC (with or without grid search), systematically outperforming the simple SQBC algorithm. This suggests that leveraging the XGBoost boosting framework was a reasonable choice. Moreover, the use of a custom SQBC loss in the XGBoost framework sometimes outperforms, not only standard XGBoost, but also the other SOTA models, CatBoost and LightGBM.

4.4 Models Efficiency

While prediction precision is an important criterion for model comparison, efficiency is also relevant. We therefore computed training time and we report the results in Table 1.

Model	Training time (s)
XGB-SQBC	0.87
LightGBM	1.11
XGBoost	2.5
SQBC	8.41
CatBoost	12.0
Grad-SQBC	9.14

Table 1: Training time in seconds for different models evaluated on synthetic dataset p=0.1%.

We can observe that in terms of training speed, LightGBM and standard XGBoost remain among the fastest. Interestingly, the integration of SQBC into the XGBoost framework (XGB-SQBC) results in training times even lower than standard XGBoost, making it the fastest model of all those tested. On the other hand, SQBC and the gradient-based adaptation of τ (Grad-SQBC) are significantly slower, but not as slow as CatBoost which is the more computationally demanding.

It is important to emphasize that SQBC and XGB-SQBC becomes more competitive when combined with grid search for τ, which was not included in this timing experiment and is time-significant. This remark is particularly true for SQBC, as Figure 5 shows that XGB-SQBC often ranks among the best-performing models even without grid search.

5 Discussion

In this report, we implemented the SQBC algorithm and applied it to the specific context of imbalanced classification. By exploring grid search and gradient descent tuning strategies, we explored whether varying the quantile level τ could improve performance. Based on PR AUC, we compared SQBC against state-of-the-art gradient boosting methods, and additionally proposed an approach which combine them with SQBC with the integration of the smooth quantile loss into the XGBoost framework. This yielded promising results.

Overall, no method really stood out as superior in handling data imbalance, with none able to maintain a high PR AUC as the proportion of positives examples decreased. SQBC showed interesting behavior, but it was not competitive enough in terms of predictive power, especially considering its slower training and the need for grid search.

In contrast, the XGB-SQBC variant appeared both fast and effective, outperforming several baseline models. It represents a strong candidate for imbalanced classification tasks.

Our main objective of adaptive SQBC on τ did not lead to significant improvements. Despite the potential advantages of SQBC with gradient descent on τ highlighted in the beginning of Section 3.5, this method presents several disadvantages:

- the loss is not necessarily convex with respect to τ. As a result, convergence is not guaranteed, and optimization may get stuck in local minima
- unlike other implemented methods, convergence strongly depends on a rigorous fine tuning of parameters such as the learning rate, the initialization

of θ and the number of estimators

- similarly to standalone SQBC, it does not include any regularization mechanism, which may lead to unstable behavior during training.

In this work, we limited gradient descent to 100 iterations. This choice was made to align with the other models but it may have restricted its potential. With more iterations and finer hyperparameter tuning, we believe this method could achieve better results. Alternatively, the complete derivation of the loss as a function of the different learned parameters could be performed outside the framework of gradient boosting using full gradient descent, as done in deep learning.

Several other directions are interesting for further work. Given the promising results of the XGB-SQBC variant, it would be worth exploring the integration of the SQBC loss within other boosting frameworks. Indeed, the current implementation of this loss performs gradient descent in a relatively straightforward manner and does not leverage the more advanced optimization techniques developed in recent years.

It could also be interesting to explore alternative asymmetric loss functions for handling class imbalance. Several options have been proposed in the literature and are discussed in Wang et al. (2022).

Finally, post-hoc optimization of the decision threshold t (fixed at 0.5 for probabilistic models and at 0 for SQBC) could help lessen several of the mentioned issues, especially for the gradient-based variant. The SQBC loss is defined to align $\mathbf{1}(f(x, \beta) \geq 0)$ with the true target Y. Therefore, a post-training adjustment breaks the strict interpretability of the model, but it allows to adjust classification outcomes in order to optimize metrics sensitive to threshold, such as recall or precision.

6 Endmatter

The different datasets used in this paper are accessible with the links below:

- sheart
- churn
- heart
- fraud (from ICL MSc statistics data science challenge)

Python code for reproducing results is available here.

ChatGPT was used for tables formatting and grammar assistance.

References

Bentéjac, C., Csörgő, A., and Martínez-Muñoz, G. (2021). A comparative analysis of gradient boosting algorithms. *Artificial Intelligence Review*, 54(3):1937–1967.

Bühlmann, P. and Yu, B. (2003). Boosting with the l 2 loss: regression and classification. *Journal of the American Statistical Association*, 98(462):324–339.

Burez, J. and Van den Poel, D. (2009). Handling class imbalance in customer churn prediction. *Expert Systems with Applications*, 36(3):4626–4636.

Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357.

Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794.

Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of computer and system sciences*, 55(1):119–139.

Friedman, J., Hastie, T., and Tibshirani, R. (2000). Additive logistic regression: a statistical view of boosting (with discussion and a rejoinder by the authors). *The annals of statistics*, 28(2):337–407.

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232.

Hernández-Orallo, J., Flach, P., and Ferri, C. (2012). A unified view of performance metrics: Translating threshold choice into expected classification loss. *The Journal of Machine Learning Research*, 13(1):2813–2869.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30.

Koenker, R. and Hallock, K. F. (2001). Quantile regression. *Journal of economic perspectives*, 15(4):143–156.

Kordas, G. (2006). Smoothed binary regression quantiles. *Journal of Applied Econometrics*, 21(3):387–407.

Murtaza, G., Shuib, L., Abdul Wahab, A. W., Mujtaba, G., Mujtaba, G., Nweke, H. F., Al-garadi, M. A., Zulfiqar, F., Raza, G., and Azmi, N. A. (2020). Deep learning-based breast cancer classification through medical imaging modalities: state of the art and research challenges. *Artificial Intelligence Review*, 53(3):1655–1720.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). Catboost: unbiased boosting with categorical features. *Advances in neural information processing systems*, 31.

Ranjan, H. (2025). Smote for class imbalance: A tool, not a fix-all. Medium. Medium blog post.

Sofaer, H. R., Hoeting, J. A., and Jarnevich, C. S. (2019). The area under the precision-recall curve as a performance metric for rare binary events. *Methods in Ecology and Evolution*, 10(4):565–577.

Wang, Q., Ma, Y., Zhao, K., and Tian, Y. (2022). A comprehensive survey of loss functions in machine learning. *Annals of Data Science*, 9(2):187–212.

Wang, Z. and Ye, W. (2016). An efficient smooth quantile boost algorithm for binary classification. *Advances in Pure Mathematics*, 6(9):615–624.

Weiss, G. M. (2004). Mining with rarity: a unifying framework. *ACM Sigkdd Explorations Newsletter*, 6(1):7–19.

West, J. and Bhattacharya, M. (2016). Intelligent financial fraud detection: a comprehensive review. *Computers & security*, 57:47–66.

Zheng, S. (2010). Boosting based conditional quantile estimation for regression and binary classification. In *Mexican International Conference on Artificial Intelligence*, pages 67–79. Springer.

Zheng, S. (2011). Gradient descent algorithms for quantile regression with smooth approximation. *International Journal of Machine Learning and Cybernetics*, 2(3):191–207.

7 Supplementary material

Synthetic (p=50%, n=10000)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
Grad M2	0.99	0.94	0.99	0.94	0.92	0.95	0.95	0.50
CatBoost	0.98	0.91	0.98	0.91	0.91	0.92	0.92	nan
XGBoost	0.97	0.89	0.97	0.89	0.89	0.90	0.90	nan
LightGBM	0.97	0.89	0.96	0.89	0.89	0.89	0.89	nan
SQBC	0.96	0.88	0.96	0.88	0.88	0.88	0.88	0.50
SQBC_gs	0.96	0.86	0.96	0.88	0.79	0.95	0.96	0.46
Grad M1	0.96	0.88	0.96	0.88	0.88	0.88	0.88	0.50
XGB-SQBC	0.95	0.87	0.95	0.87	0.86	0.88	0.88	0.50
XGB-SQBC_gs	0.95	0.70	0.95	0.77	0.55	0.97	0.98	0.40

Table 2: Results of all models on synthetic dataset (p=50%)

Synthetic (p=10%, n=10000)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
Grad M2	0.96	0.89	1.00	0.98	0.90	0.88	0.99	0.52
CatBoost	0.92	0.78	0.99	0.96	0.68	0.93	0.99	nan
XGBoost	0.90	0.76	0.99	0.96	0.66	0.89	0.99	nan
LightGBM	0.90	0.72	0.99	0.96	0.58	0.96	1.00	nan
XGB-SQBC	0.85	0.62	0.98	0.94	0.46	0.93	1.00	0.50
XGB-SQBC_gs	0.85	0.65	0.98	0.95	0.50	0.94	1.00	0.52
SQBC_gs	0.82	0.74	0.98	0.95	0.76	0.72	0.97	0.57
Grad M1	0.82	0.64	0.97	0.94	0.51	0.86	0.99	0.53
SQBC	0.81	0.53	0.97	0.93	0.38	0.89	1.00	0.50

Table 3: Results of all models on synthetic dataset with $p = 10\%$

Synthetic (p=5%, n=10000)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
Grad M2	0.97	0.87	1.00	0.99	0.79	0.96	1.00	0.52
CatBoost	0.87	0.64	0.99	0.97	0.48	0.96	1.00	nan
LightGBM	0.82	0.53	0.99	0.97	0.37	0.95	1.00	nan
XGBoost	0.77	0.59	0.98	0.97	0.45	0.87	1.00	nan
XGB-SQBC_gs	0.73	0.45	0.98	0.96	0.31	0.84	1.00	0.52
SQBC_gs	0.72	0.56	0.98	0.97	0.43	0.81	0.99	0.58
Grad M1	0.70	0.27	0.97	0.96	0.16	0.89	1.00	0.53
SQBC	0.66	0.15	0.96	0.95	0.08	1.00	1.00	0.50
XGB-SQBC	0.65	0.47	0.98	0.96	0.33	0.80	1.00	0.50

Table 4: Results of all models on synthetic dataset (p=5%)

Synthetic (p=1%, n=10000)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
Grad M2	0.78	0.00	1.00	0.99	0.00	0.00	1.00	0.52
CatBoost	0.62	0.26	0.99	0.99	0.15	1.00	1.00	nan
LightGBM	0.55	0.18	0.98	0.99	0.10	1.00	1.00	nan
XGB-SQBC	0.53	0.17	0.98	0.99	0.10	0.67	1.00	0.50
XGBoost	0.45	0.16	0.98	0.99	0.10	0.40	1.00	nan
XGB-SQBC_gs	0.35	0.08	0.98	0.99	0.05	0.25	1.00	0.52
SQBC_gs	0.20	0.02	0.97	0.01	1.00	0.01	0.00	0.81
Grad M1	0.14	0.00	0.94	0.99	0.00	0.00	1.00	0.53
SQBC	0.08	0.00	0.80	0.99	0.00	0.00	1.00	0.50

Table 5: Results of all models on synthetic dataset (p=1%)

Synthetic (p=0.1%, n=10000)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
Grad M2	0.16	0.00	1.00	1.00	0.00	0.00	1.00	0.52
CatBoost	0.05	0.00	0.98	1.00	0.00	0.00	1.00	nan
XGB-SQBC	0.02	0.00	0.95	1.00	0.00	0.00	1.00	0.50
XGB-SQBC_gs	0.02	0.00	0.95	0.00	1.00	0.00	0.00	0.87
XGBoost	0.01	0.00	0.90	1.00	0.00	0.00	1.00	nan
LightGBM	0.00	0.00	0.17	0.99	0.00	0.00	0.99	nan
SQBC	0.00	0.00	0.49	1.00	0.00	0.00	1.00	0.50
SQBC_gs	0.00	0.00	0.23	1.00	0.00	0.00	1.00	0.64
Grad M1	0.00	0.00	0.48	1.00	0.00	0.00	1.00	0.53

Table 6: Results of all models on synthetic dataset (p=0.1%)

Sheart (p=54.5%, n=303)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
SQBC_gs	0.93	0.79	0.92	0.79	0.73	0.86	0.86	0.40
Grad M2	0.90	0.90	0.90	0.89	0.94	0.86	0.82	0.50
XGBoost	0.88	0.81	0.89	0.80	0.79	0.84	0.82	nan
LightGBM	0.88	0.82	0.89	0.80	0.85	0.80	0.75	nan
CatBoost	0.88	0.88	0.90	0.87	0.88	0.88	0.86	nan
SQBC	0.86	0.83	0.91	0.82	0.82	0.84	0.82	0.50
Grad M1	0.86	0.84	0.91	0.84	0.82	0.87	0.86	0.50
XGB-SQBC	0.83	0.87	0.87	0.85	0.88	0.85	0.82	0.50
XGB-SQBC_gs	0.82	0.81	0.86	0.80	0.79	0.84	0.82	0.46

Table 7: Results of all models on real dataset `Sheart`

Churn (p=20.4%, n=9799)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
XGBoost	0.23	0.09	0.52	0.77	0.06	0.26	0.96	nan
LightGBM	0.21	0.01	0.51	0.79	0.00	0.33	1.00	nan
CatBoost	0.21	0.00	0.50	0.80	0.00	0.00	1.00	nan
SQBC	0.21	0.00	0.52	0.80	0.00	0.00	1.00	0.50
XGB-SQBC_gs	0.21	0.34	0.51	0.21	1.00	0.20	0.00	0.81
Grad M1	0.21	0.00	0.52	0.80	0.00	0.00	1.00	0.48
Grad M2	0.21	0.00	0.51	0.80	0.00	0.00	1.00	0.46
SQBC_gs	0.20	0.34	0.50	0.20	1.00	0.20	0.00	0.81
XGB-SQBC	0.19	0.01	0.49	0.79	0.00	0.29	1.00	0.50

Table 8: Results of all models on real dataset `Churn`

Heart (p=5%, n=10000)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
SQBC_gs	0.20	0.02	0.82	0.95	0.01	0.25	1.00	0.52
Grad M1	0.20	0.02	0.82	0.95	0.01	0.25	1.00	0.52
Grad M2	0.20	0.04	0.82	0.95	0.02	0.50	1.00	0.51
SQBC	0.19	0.00	0.82	0.95	0.00	0.00	1.00	0.50
XGB-SQBC	0.19	0.04	0.82	0.95	0.02	0.25	1.00	0.50
LightGBM	0.17	0.08	0.82	0.94	0.05	0.25	0.99	nan
XGB-SQBC_gs	0.17	0.10	0.80	0.05	1.00	0.05	0.00	0.93
CatBoost	0.16	0.08	0.80	0.94	0.05	0.25	0.99	nan
XGBoost	0.15	0.09	0.80	0.94	0.06	0.17	0.98	nan

Table 9: Results of all models on real dataset `Heart`

Fraud (p=0.1%, n=80000)								
Model	**pr_auc**	**f1**	**roc_auc**	**acc.**	**recall**	**prec.**	**spec.**	τ
CatBoost	0.56	0.43	1.00	1.00	0.33	0.60	1.00	nan
XGB-SQBC	0.51	0.67	1.00	1.00	0.56	0.83	1.00	0.50
XGBoost	0.44	0.43	1.00	1.00	0.33	0.60	1.00	nan
XGB-SQBC_gs	0.43	0.00	0.99	0.00	1.00	0.00	0.00	0.81
SQBC_gs	0.34	0.00	0.99	0.00	1.00	0.00	0.00	0.81
SQBC	0.31	0.33	0.83	1.00	0.22	0.67	1.00	0.50
Grad M1	0.30	0.33	0.83	1.00	0.22	0.67	1.00	0.53
LightGBM	0.00	0.03	0.32	1.00	0.11	0.02	1.00	nan

Table 10: Results of all models except "Grad M2" on real dataset `Fraud`